

Toollery: Self-Verified Query Retrieval for Scalable Tool and Skill Selection

Anonymous ACL submission

Abstract

LLM agents increasingly choose among large libraries of skills, where each skill may be a reusable procedure or an *atomic skill* such as a tool/API with a name, description, and schema. Prompting the LLM with all candidates scales poorly in tokens, latency, and accuracy. We present **Toollery**, a self-verified query retrieval framework for scalable skill selection. Offline, Toollery generates user-like candidate queries for each skill and keeps only those that pass round-trip self-verification: a verifier must recover the source skill from a pool containing distractors. The retained queries form a verified query index. Online, the user request retrieves similar verified queries, their linked skills are aggregated into a compact candidate subset, and only compact skill specifications are passed to the final LLM for selection and argument generation. Thus generated queries improve retrieval but never enter the final prompt. Toollery decouples large-library search from LLM decision making, reducing candidate tokens and latency while preserving the LLM’s schema-aware final choice.

1 Introduction

Large language models (LLMs) are increasingly used as agents that invoke external capabilities, including APIs, software tools, and reusable skills (Schick et al., 2023; Qin et al., 2023; Patil et al., 2024; Yao et al., 2022a). We use *skill* as the general unit of selection, and *atomic skill* for tool-style APIs described by a name, natural-language description, schema, and optional examples or manual. In both cases, the agent must identify the relevant capability before it can generate a valid invocation.

This selection step becomes a scalability bottleneck as skill libraries grow. Placing every candidate specification in the prompt increases cost and latency, can exceed context limits, and asks the LLM

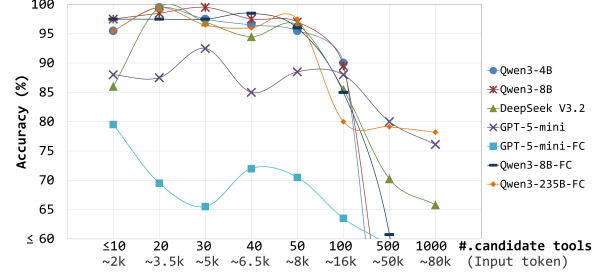


Figure 1: Impact of candidate pool size on tool selection accuracy in our ScaleTool-style atomic-tool analysis. Accuracy degrades as the number of available tools grows from 10 to 1,000, motivating candidate compression before final LLM selection.

to compare many irrelevant or semantically adjacent options. Even when the full library fits, long candidate lists can distract the model and hurt selection accuracy (Paramanayakam et al., 2025). The candidate-pool analysis in Figure 1 illustrates this degradation as the number of available atomic tools grows. Practical assistants that expose operating-system actions, enterprise APIs, cloud services, retrieval modules, and domain skills therefore need a way to narrow the candidate set before LLM inference (Wu et al., 2024; Amazon Web Services, 2024; Mo et al., 2025; Xu et al., 2025).

A natural solution is retrieval: retrieve candidate skills from their names, descriptions, or documentation, then ask the LLM to choose among the subset (Agarwal et al., 2025; Lin et al., 2025; Zheng et al., 2026). However, direct query-to-metadata retrieval suffers from a semantic mismatch. Users express goals in contextual language, while skill specifications often describe implementation-facing capabilities. For example, a user may ask whether to bring an umbrella to an afternoon meeting, while the relevant atomic skill is specified as `get_hourly_forecast(location, time)`. The intent and the metadata are aligned functionally, but not lexically.

We introduce **Toollery**, a framework for **self-verified query retrieval** for scalable skill and atomic-skill selection. Toollery builds an intent layer around each skill rather than retrieving directly over static metadata. Offline, a generator LLM writes diverse candidate queries that resemble real user requests for each skill. A verifier LLM then performs a round-trip check: given a generated query and a candidate pool containing the source skill plus distractors, it must recover the source skill. Only queries that pass this check are retained as *verified queries* in a verified query index.

At inference time, Toollery retrieves from this verified query index, aggregates query-level matches into skill-level scores, and constructs a compact top- k candidate subset. The final LLM receives the original user request and only the compact specifications of these candidates; verified queries are used solely for retrieval and are not included in the final prompt. This design keeps the final decision schema-aware while reducing prompt tokens, LLM prefill cost, and latency from scaling with the full library size to scaling with the chosen candidate subset.

Our contribution is a compact candidate-compression layer for LLM agents: (i) we formulate scalable skill selection as retrieval over self-verified user-intent queries; (ii) we construct the index through generation followed by round-trip verification against distractors; and (iii) we expose a simple interface that sends only compact candidate skill specifications to the final LLM, making Toollery compatible with function-calling models, routers, and skill-selection pipelines without fine-tuning.

2 Related Work

Tool-augmented LLMs. Tool-augmented language models extend LLMs from text generation to action generation by allowing them to call external APIs, software interfaces, search engines, calculators, and reusable agent skills. Prior work studies how models learn to invoke tools, follow schemas, and generate valid calls (Schick et al., 2023; Yao et al., 2022b; Patil et al., 2024; Qin et al., 2023; Li et al., 2023; Liu et al., 2024a,b). Toollery is complementary: it does not train a stronger function-calling model, but reduces the candidate action space that the model must inspect before making a call.

Tool and skill retrieval. A growing line of work treats tool selection as retrieval or routing rather than full-library prompting. ToolRM, MassTool, RAG-MCP, and context-pruning methods such as Less is More all aim to expose the LLM to a compact and relevant subset of tools (Agarwal et al., 2025; Lin et al., 2025; Gan and Sun, 2025; Paramanayakam et al., 2025). SkillRouter is especially related because it proposes a compact full-text retrieve-and-rerank pipeline for large skill registries (Zheng et al., 2026). Toollery makes a different design choice: it constructs an offline index of self-verified generated usage queries, then maps retrieved intents back to compact skill specifications. It can therefore serve as a pre-routing candidate compression layer before a standard LLM selector, function-calling interface, or SkillRouter-style reranker.

Query augmentation and RAG. Toollery is connected to query augmentation methods that generate pseudo-queries or synthetic intents to reduce vocabulary mismatch in retrieval. In tool and skill selection, the mismatch is between user goals and developer-facing metadata. Toollery generates user-like queries for each tool or skill, then retains only those that pass round-trip verification against distractors. Generic RAG systems, including RAGAnything-style pipelines (Guo et al., 2025), retrieve documentation chunks and condition the LLM on retrieved text. Toollery instead retrieves verified intent anchors, maps them back to skill identifiers, and sends only compact candidate specifications to the final LLM. The goal is candidate compression for constrained selection, not open-ended document grounding.

3 Method

Toollery separates large-library search from final LLM decision making. Figure 2 shows the two-stage pipeline: offline verified query index construction and online verified query retrieval. Given a skill library $\mathcal{A} = \{a_1, \dots, a_M\}$, each skill a_i has a specification s_i containing fields such as name, description, schema, examples, or skill instructions. For a user request x , the goal is to retrieve a small candidate set $\mathcal{C}_k(x) \subset \mathcal{A}$ that contains the needed skill while satisfying $|\mathcal{C}_k(x)| \ll M$. The downstream LLM then receives x and compact specifications for $\mathcal{C}_k(x)$, and produces the final selected skill and arguments.

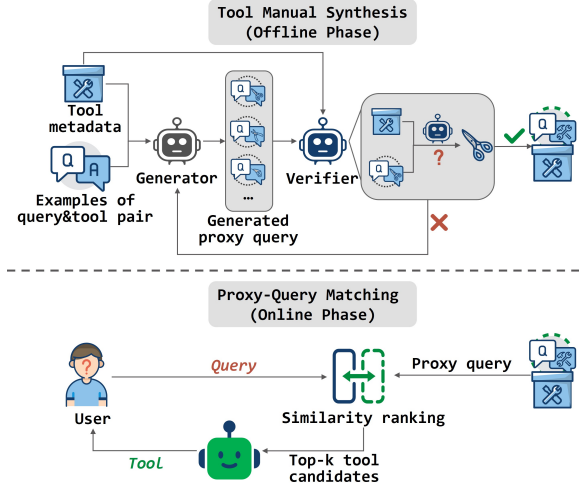


Figure 2: Overview of Toollery. Offline synthesis constructs a verified query index, and online proxy-query matching retrieves verified intents that are aggregated into a compact candidate set for final LLM selection.

3.1 Verified Query Index

Toollery builds an offline verified query index rather than prompting over all specifications. For each skill a_i , a generator model G produces candidate usage queries $Q_i = G(s_i; N)$, where N is the target number of generated queries. These queries are retrieval anchors, not final-prompt content: they translate skill-centric metadata into user-centric intents. For atomic skills, the generator sees the tool/API name, description, schema, return information when available, and examples; for higher-level skills, it sees the capability description, triggering conditions, inputs, and constraints.

Generated queries are noisy: some are too broad, ambiguous, or inconsistent with the source skill. Toollery filters them with round-trip self-verification. For each $q \in Q_i$, it constructs a pool

$$\mathcal{P}_{i,q} = \{a_i\} \cup \mathcal{D}_{i,q}, \quad (1)$$

where $\mathcal{D}_{i,q}$ contains random or hard distractor skills. A verifier model H receives q and compact specifications for the skills in $\mathcal{P}_{i,q}$, then selects the best matching skill or abstains. The query is retained only if the verifier recovers the source:

$$V_i = \{q \in Q_i : H(q, \{s_j : a_j \in \mathcal{P}_{i,q}\}) = a_i\}. \quad (2)$$

The full index is

$$\mathcal{I} = \{(v, a_i) : v \in V_i, a_i \in \mathcal{A}\}. \quad (3)$$

Each entry stores a verified natural-language intent, its representation for retrieval, and the identifier of

its source skill. The full skill specification remains outside this index and is retrieved only after skill aggregation. This distinction is central: Toollery indexes generated text for search, but the final LLM prompt contains only compact candidate skill specifications.

3.2 Retrieval and LLM Selection

At inference time, Toollery embeds the user request and verified queries into a shared retrieval space. For each index entry (q_i, a_i) , it computes $r_i = \cos(e(x), e(q_i))$, retrieves the top- m verified queries, and aggregates query-level scores into skill-level scores. A simple aggregation is

$$R(a | x) = \max_{q_i : a_i = a} r_i, \quad (4)$$

though mean, log-sum-exp, reciprocal-rank fusion, or verification-weighted variants can also be used. Aggregation converts multiple verified intents for the same skill into a single ranked candidate list. Toollery then selects

$$\mathcal{C}_k(x) = \text{TopK}_{a \in \mathcal{A}} R(a | x), \quad (5)$$

instantiates compact specifications for these k skills, and sends them with the original user request to the final LLM. Retrieval is therefore not the final tool call; it only reduces the action space. The LLM remains responsible for resolving ambiguity, selecting the skill, and filling arguments.

This design changes the online cost from full-library prompting to subset prompting. Figure 3 contrasts the two inference patterns. If L_s is the average token length of a skill specification, uncompressed prompting sends approximately $O(M \cdot L_s)$ skill-definition tokens to the LLM. Toollery sends approximately $O(k \cdot L_s)$ tokens, where $k \ll M$ and can remain fixed as the library grows. The verified query index grows offline, but generation and verification are amortized, and online vector retrieval replaces a much larger LLM-context expansion. Consequently, Toollery targets both quality and efficiency: high candidate recall under large skill pools, with lower final-prompt tokens and LLM latency.

4 Experiments

We evaluate Toollery on two complementary settings: broad skill routing and atomic tool/API selection. In this paper, tools and APIs are treated

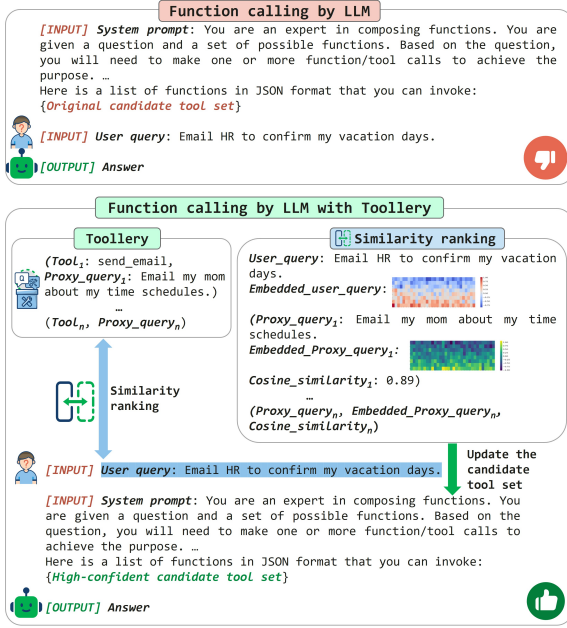


Figure 3: Comparison between full-context tool selection and Toollery-style candidate compression. Toollery retrieves a compact subset before final LLM inference, reducing prompt size while preserving schema-aware selection.

as atomic skills, while higher-level skills may include longer instructions, documentation, and triggering conditions. Table 1 defines the three method settings used throughout the experiments. A *skill* denotes a reusable capability described by natural-language instructions or documentation; an *atomic skill* denotes a tool-style API unit with a name, description, schema, and optional manual. Generated and verified queries are used only inside the retrieval index. The final LLM receives the user request and compact specifications for the retrieved top- k candidates, not the generated query pool.

Benchmarks. The first track evaluates skill routing on SkillBench-style tasks aligned with SkillRouter; its result layout is shown in Table 2. The second track evaluates atomic tool / atomic skill retrieval on candidate pools with gold tool labels, as shown in Table 3. Separately, we report end-to-end atomic-tool selection accuracy on HammerBench, xLAM, and BFCL under increasing candidate-pool sizes. Across both retrieval and end-to-end evaluations, we vary native and expanded candidate pools with random and hard distractors, e.g., $N \in \{100, 500, 1000, 5000, 10000\}$ when data permit.

Method settings and baselines. For retrieval/routing, we compare against BM25 and dense retrieval over raw skill text, a SkillRouter or SkillRouter-style retrieve-and-rerank pipeline on SkillBench, and RAGAnything adapted as a retrieval-over-documentation baseline. For end-to-end and efficiency comparisons, we also include direct all-candidate LLM selection where the context budget allows. Each retrieval baseline is evaluated in a raw setting and a query-augmented setting that appends the same Toollery-generated candidate queries to the searchable document. Unlike Full Toollery, these query-augmented baselines search over augmented documents rather than a verified query-level index. The main comparison therefore separates raw-document retrieval, query-augmented retrieval, and Full Toollery.

Metrics and efficiency. For retrieval, we report Hit@ k for single-gold tasks and Recall@ k /nDCG@ k for multi-gold tasks, with MRR as a ranking metric. For single-gold atomic-tool tasks, Recall@ k reduces to Hit@ k ; FullCoverage@ k is used only when multiple required tools or skills must all appear in the candidate set. Candidate-set recall before final LLM selection is reported separately from final success. For end-to-end performance, we use exact match, pass rate, final task success, or each benchmark’s official score. For offline construction, we report generated queries per skill, verification acceptance rate, verified queries per skill, zero-query skill rate, construction latency, and indexing cost.

Efficiency is a first-class metric because Toollery’s claim is not only better retrieval but also cheaper final selection. Table 4 summarizes the token and latency compression measurements. We measure candidates sent to the final LLM, prompt tokens, completion tokens, retrieval latency, reranking latency, final LLM latency, total latency, and estimated cost per 1,000 queries. Compression is reported relative to direct all-candidate prompting:

$$\text{TokenComp.} = \frac{\text{PromptTokens}_{\text{all}}}{\text{PromptTokens}_{\text{method}}}, \quad (6)$$

$$\text{LatencyComp.} = \frac{\text{LLMLatency}_{\text{all}}}{\text{LLMLatency}_{\text{method}}}. \quad (7)$$

When all-candidate prompting is infeasible for large pools, we use the largest feasible pool as the

reference and mark larger-pool results as retrieval-only or retrieval-plus-LLM-subset evaluation.

End-to-end atomic-tool selection results. In addition to the unified retrieval tables above, we report end-to-end atomic-tool selection experiments that evaluate Toollery candidate compression under different final LLM backbones. Tables 5 and 6 report end-to-end selection accuracy on ScaleTool-HammerBench and ScaleTool-xLAM under increasing candidate-pool sizes. Tables 7 and 8 report BFCL-V4 non-live multiple and live multiple results. These tables evaluate downstream LLM tool selection after candidate compression, whereas Tables 2 and 3 isolate retrieval/routing quality before final LLM selection.

5 Discussion and Limitations

Toollery’s claim is intentionally narrow: it is not a universal skill router, a replacement for trained function-calling models, or a generic RAG system over tool documentation. It is a self-verified generated-query retrieval layer that compresses a large candidate library before final LLM selection. This framing explains why Toollery can be combined with stronger routers or selectors: retrieval reduces the search space, while the final model still performs schema-aware selection and argument generation.

The comparison with SkillRouter and RAGAnything clarifies the boundary. SkillRouter retrieves and reranks over skill text; RAGAnything retrieves documentation chunks for grounding; Toollery retrieves verified user-intent anchors and maps them back to compact candidate specifications. If Toollery improves under fixed k , the gain suggests that verified intents better align retrieval with how users express tasks. If it mainly improves token and latency cost while matching accuracy, it still serves as an effective candidate-compression mechanism for large tool libraries.

Toollery depends on the coverage and precision of its verified query index. If generation misses realistic user intents, retrieval recall is bounded; if verification uses weak distractors, ambiguous queries may enter the index. The method also adds offline maintenance cost: new or modified skills require query generation, verification, embedding, and index updates. This cost is amortized across future requests but matters in rapidly changing tool ecosystems. Finally, Toollery primarily addresses single-step candidate selection. It does not solve

multi-tool planning, execution monitoring, recovery from failed calls, or argument correctness by itself.

6 Conclusion

We presented Toollery, a self-verified generated-query retrieval framework for scalable tool and skill selection. Toollery moves part of semantic grounding offline by generating user-like queries for each action and retaining only those that pass round-trip verification against distractors. At inference time, the user request retrieves verified queries, aggregates them into a compact candidate subset, and passes only top- k tool or skill specifications to the final LLM. This positions Toollery as a pre-routing candidate compression layer that aims to preserve or improve selection quality while reducing prompt tokens, latency, and distractor pressure as tool and skill libraries grow.

Setting	Searchable text	Generated queries	Retrieval unit	Final LLM input	Purpose
Raw Baseline	Original skill/tool fields only	Not used	Raw document	Top- k compact specs	Raw retrieval level
Query-Augmented Baseline	Original fields plus shared generated queries	Appended to searchable documents	Augmented document	Top- k compact specs	Effect of generated queries
Full Toollery	Original fields used offline to construct verified query index	Generated and self-verified	Verified query	Top- k compact specs	Verified intent retrieval and compression; dense retrieval in main experiments

Table 1: Method setting definitions for candidate retrieval/routing. The searchable text differs across settings, while the final LLM, when evaluated, receives only the original request and compact top- k candidate specifications. Generated or verified queries are used for retrieval/indexing and are not copied into the final LLM prompt.

Setting	Method	Hit@1	Recall@10	Recall@50	MRR@10	nDCG@10	FullCoverage@10
Raw Baseline	BM25 [†]	.314	.321	.433	.365	—	—
Raw Baseline	Dense (Qwen3-Emb-8B) [†]	.640	.659	.790	.698	—	—
Raw Baseline	RAGAnything	.600	.568	.712	.663	.500	.393
Raw Baseline	SkillRouter [†]	.740	.704	—	.791	—	.520
Query-Augmented Baseline	BM25	.920	.942	.968	.942	.880	.853
Query-Augmented Baseline	Dense	.587	.572	.719	.664	.510	.393
Query-Augmented Baseline	RAGAnything	.587	.572	.719	.664	.510	.393
Full Toollery	Dense verified-query retrieval	.933	.933	.957	.951	.890	.867

Table 2: SkillRouter / SkillBench-compatible retrieval results. Non-daggers values are from our unified full-pool v2 runs and average the easy and hard splits. [†] denotes reference values directly reported in the SkillRouter paper.

Setting	Method	Hit@1	Recall@10	Recall@50	MRR@10	nDCG@10	FullCoverage@10
Raw Baseline	BM25	.840	.975	.991	.890	.911	.975
Raw Baseline	Dense	.903	.993	.999	.940	.953	.993
Raw Baseline	RAGAnything	.903	.993	.999	.940	.953	.993
Query-Augmented Baseline	BM25	.885	.982	.995	.922	.937	.982
Query-Augmented Baseline	Dense	.917	.994	.999	.949	.960	.994
Query-Augmented Baseline	RAGAnything	.917	.994	.999	.949	.960	.994
Full Toollery	Dense verified-query retrieval	.868	.979	.990	.911	.927	.979

Table 3: Atomic tool / atomic skill v2 retrieval quality on BFCL-live scaled instances. Values are computed from top-50 retrieval-quality runs. Recall@ k is the fraction of gold tools retrieved in the top- k candidates; for single-gold cases it is equivalent to Hit@ k .

Setting	Method	Pool size	Candidates to LLM	Prompt tokens	Retr. latency	Total latency
Reference	Full-context LLM			<i>not run for full candidate pools</i>		
Raw Baseline	BM25 subset	457	3	625.9	6.0 ms	6.0 ms
Raw Baseline	Dense subset	457	3	606.0	171.8 ms	172.0 ms
Raw Baseline	RAGAnything subset	457	3	606.0	113.9 ms	114.0 ms
Query-Augmented Baseline	BM25 + generated queries	457	3	612.5	9.4 ms	9.4 ms
Query-Augmented Baseline	Dense + generated queries	457	3	610.8	82.1 ms	82.2 ms
Query-Augmented Baseline	RAGAnything + generated queries	457	3	610.8	139.8 ms	140.0 ms
Full Toollery	Dense verified-query retrieval	457	3	614.3	103.0 ms	103.0 ms

Table 4: Token and latency measurements from the completed BFCL-live v2 atomic-tool subset-selection runs. Values are averaged per query after retrieving three candidate tools. Full-context prompting over the complete candidate pool is not run here because it is substantially more expensive and can exceed practical context budgets for larger pools.

Table 5: End-to-end atomic-tool selection accuracy on ScaleTool-HammerBench under increasing candidate-pool sizes. These results measure downstream LLM selection after candidate compression, not standalone retrieval quality. Prompt and FC denote prompt-based and function-calling selection, respectively.

Model	≤ 10	20	30	40	50	100	500	1000
Qwen3-4B (Prompt)	49.67	55.60	52.20	45.00	55.20	50.30	31.25	\
Qwen3-4B (Prompt) + Toollery	72.67 $\uparrow$	64.20 $\uparrow$	64.60 $\uparrow$	69.60 $\uparrow$	68.40 $\uparrow$	64.15 $\uparrow$	67.50 $\uparrow$	66.50 $\uparrow$
Qwen3-8B (Prompt)	72.33	76.20	73.40	72.20	75.40	60.20	56.25	\
Qwen3-8B (Prompt) + Toollery	72.00	76.40 $\uparrow$	71.40	82.20 $\uparrow$	80.40 $\uparrow$	75.50 $\uparrow$	62.50 $\uparrow$	62.00 $\uparrow$
DeepSeek V3.2 (Prompt)	97.00	77.20	75.20	81.80	77.40	75.00	70.00	63.00
DeepSeek V3.2 (Prompt) + Toollery	96.67	86.60 $\uparrow$	88.20 $\uparrow$	89.80 $\uparrow$	82.40 $\uparrow$	89.85 $\uparrow$	85.75 $\uparrow$	78.00 $\uparrow$
GPT-5-mini (Prompt)	89.00	84.20	85.80	83.60	90.40	70.00	62.50	54.00
GPT-5-mini (Prompt) + Toollery	89.67 $\uparrow$	85.00 $\uparrow$	87.80 $\uparrow$	84.40 $\uparrow$	95.00 $\uparrow$	85.50 $\uparrow$	78.00 $\uparrow$	78.00 $\uparrow$
GPT-5-mini (FC)	40.33	44.60	27.80	31.20	22.40	10.00	2.50	\
GPT-5-mini (FC) + Toollery	44.67 $\uparrow$	47.20 $\uparrow$	44.60 $\uparrow$	51.20 $\uparrow$	44.40 $\uparrow$	45.50 $\uparrow$	42.95 $\uparrow$	44.40 $\uparrow$
Qwen3-8B (FC)	69.67	74.20	73.60	68.60	67.00	61.50	30.50	\
Qwen3-8B (FC) + Toollery	91.33 $\uparrow$	86.40 $\uparrow$	89.00 $\uparrow$	84.80 $\uparrow$	82.20 $\uparrow$	85.50 $\uparrow$	75.80 $\uparrow$	77.50 $\uparrow$
Qwen3-235B (FC)	73.67	73.40	64.60	63.00	64.80	35.00	11.25	\
Qwen3-235B (FC) + Toollery	86.33 $\uparrow$	85.40 $\uparrow$	84.60 $\uparrow$	82.20 $\uparrow$	76.40 $\uparrow$	82.50 $\uparrow$	75.00 $\uparrow$	70.50 $\uparrow$

Table 6: End-to-end atomic-tool selection accuracy on ScaleTool-xLAM under increasing candidate-pool sizes. These results measure downstream LLM selection after candidate compression, not standalone retrieval quality.

Model	≤ 10	20	30	40	50	100	500	1000
Qwen3-4B (Prompt)	86.33	92.00	84.40	87.20	82.00	61.59	39.88	\
Qwen3-4B (Prompt) + Toollery	88.00 $\uparrow$	92.00	88.80 $\uparrow$	91.40 $\uparrow$	91.40 $\uparrow$	90.71 $\uparrow$	81.25 $\uparrow$	72.00 $\uparrow$
Qwen3-8B (Prompt)	90.33	83.00	87.20	88.60	80.40	66.31	41.25	\
Qwen3-8B (Prompt) + Toollery	88.33	89.80 $\uparrow$	91.20 $\uparrow$	88.20	90.00 $\uparrow$	90.00 $\uparrow$	87.50 $\uparrow$	78.95 $\uparrow$
DeepSeek V3.2 (Prompt)	88.67	93.60	95.00	92.60	89.20	85.00	83.75	16.56
DeepSeek V3.2 (Prompt) + Toollery	90.67 $\uparrow$	94.40 $\uparrow$	93.20	92.80 $\uparrow$	96.00 $\uparrow$	90.00 $\uparrow$	86.25 $\uparrow$	79.47 $\uparrow$
GPT-5-mini (Prompt)	88.67	92.00	92.80	92.00	89.40	84.13	80.50	71.59
GPT-5-mini (Prompt) + Toollery	91.67 $\uparrow$	91.40	95.40 $\uparrow$	91.40	90.20 $\uparrow$	89.56 $\uparrow$	81.50 $\uparrow$	79.64 $\uparrow$
GPT-5-mini (FC)	91.00	89.40	94.00	89.80	87.20	70.00	42.50	11.37
GPT-5-mini (FC) + Toollery	91.33 $\uparrow$	91.80 $\uparrow$	93.20	91.60 $\uparrow$	88.80 $\uparrow$	90.51 $\uparrow$	73.75 $\uparrow$	69.86 $\uparrow$
Qwen3-8B (FC)	89.33	86.20	90.60	92.40	86.00	68.28	47.50	\
Qwen3-8B (FC) + Toollery	90.67 $\uparrow$	90.80 $\uparrow$	93.60 $\uparrow$	90.20	90.40 $\uparrow$	91.06 $\uparrow$	81.25 $\uparrow$	73.54 $\uparrow$
Qwen3-235B (FC)	80.00	91.40	93.80	92.40	89.40	87.16	76.25	33.21
Qwen3-235B (FC) + Toollery	90.00 $\uparrow$	94.80 $\uparrow$	91.60	92.60 $\uparrow$	92.20 $\uparrow$	90.83 $\uparrow$	81.25 $\uparrow$	77.19 $\uparrow$

Table 7: End-to-end BFCL-V4 non-live multiple tool-selection accuracy under increasing candidate-pool sizes. These results measure downstream LLM selection after candidate compression, not standalone retrieval quality.

Model	≤ 10	20	30	40	50	100	500	1000
Qwen3-4B (Prompt)	95.50	99.50	97.50	96.50	95.50	90.00	24.25	\
Qwen3-4B (Prompt) + Toollery	100.00 $\uparrow$	97.50	98.00 $\uparrow$	95.00	98.00 $\uparrow$	91.50 $\uparrow$	83.63 $\uparrow$	81.50 $\uparrow$
Qwen3-8B (Prompt)	97.50	98.50	99.50	97.50	97.00	89.50	37.63	\
Qwen3-8B (Prompt) + Toollery	99.50 $\uparrow$	100.00 $\uparrow$	97.50	96.00	98.00 $\uparrow$	91.00 $\uparrow$	80.63 $\uparrow$	80.40 $\uparrow$
DeepSeek V3.2 (Prompt)	86.00	99.50	97.00	94.50	96.50	85.50	70.25	65.80
DeepSeek V3.2 (Prompt) + Toollery	97.50 $\uparrow$	97.50	95.00	98.00 $\uparrow$	98.50 $\uparrow$	89.50 $\uparrow$	81.38 $\uparrow$	80.80 $\uparrow$
GPT-5-mini (Prompt)	88.00	87.50	92.50	85.00	88.50	88.00	80.00	76.10
GPT-5-mini (Prompt) + Toollery	88.00	89.50 $\uparrow$	91.50	86.50 $\uparrow$	95.00 $\uparrow$	90.00 $\uparrow$	82.00 $\uparrow$	81.00 $\uparrow$
GPT-5-mini (FC)	79.50	69.50	65.50	72.00	70.50	63.50	58.63	50.50
GPT-5-mini (FC) + Toollery	81.50 $\uparrow$	77.00 $\uparrow$	75.50 $\uparrow$	73.00 $\uparrow$	76.00 $\uparrow$	69.50 $\uparrow$	68.63 $\uparrow$	68.50 $\uparrow$
Qwen3-8B (FC)	97.50	97.50	97.50	98.50	96.00	85.00	60.75	\
Qwen3-8B (FC) + Toollery	99.50 $\uparrow$	96.50	97.50	96.50	97.00 $\uparrow$	90.50 $\uparrow$	86.75 $\uparrow$	83.30 $\uparrow$
Qwen3-235B (FC)	95.50	99.50	96.50	96.00	97.50	80.00	79.13	78.20
Qwen3-235B (FC) + Toollery	96.00 $\uparrow$	99.00	98.50 $\uparrow$	98.00 $\uparrow$	98.00 $\uparrow$	93.50 $\uparrow$	94.00 $\uparrow$	86.30 $\uparrow$

Table 8: End-to-end BFCL-V4 live multiple tool-selection accuracy under increasing candidate-pool sizes. These results measure downstream LLM selection after candidate compression, not standalone retrieval quality.

Model	≤ 10	20	30	40	50	100	500	1000
Qwen3-4B (Prompt)	85.50	86.00	88.00	82.50	78.00	69.50	24.88	\
Qwen3-4B (Prompt) + Toollery	87.00 $\uparrow$	84.50	87.00	86.50 $\uparrow$	82.50 $\uparrow$	81.00 $\uparrow$	83.75 $\uparrow$	81.20 $\uparrow$
Qwen3-8B (Prompt)	97.00	93.50	92.00	84.00	83.00	84.50	38.50	\
Qwen3-8B (Prompt) + Toollery	95.50	98.50 $\uparrow$	98.50 $\uparrow$	92.50 $\uparrow$	94.50	86.50 $\uparrow$	79.38 $\uparrow$	80.30 $\uparrow$
DeepSeek V3.2 (Prompt)	94.50	80.00	84.00	89.50	79.50	63.00	77.00	73.80
DeepSeek V3.2 (Prompt) + Toollery	98.00 $\uparrow$	78.50	94.50 $\uparrow$	99.50 $\uparrow$	89.50 $\uparrow$	89.00 $\uparrow$	83.13 $\uparrow$	85.50 $\uparrow$
GPT-5-mini (Prompt)	83.00	93.00	90.00	87.00	88.00	88.00	79.75	75.20
GPT-5-mini (Prompt) + Toollery	83.50 $\uparrow$	94.00 $\uparrow$	89.50	95.00 $\uparrow$	89.50 $\uparrow$	89.00 $\uparrow$	84.50 $\uparrow$	78.80 $\uparrow$
GPT-5-mini (FC)	67.00	62.00	63.50	62.00	61.00	63.50	57.00	50.50
GPT-5-mini (FC) + Toollery	74.50 $\uparrow$	67.50 $\uparrow$	69.00 $\uparrow$	69.50 $\uparrow$	67.00 $\uparrow$	67.50 $\uparrow$	66.00 $\uparrow$	65.30 $\uparrow$
Qwen3-8B (FC)	93.50	94.50	92.50	83.00	83.50	83.00	60.00	15.90
Qwen3-8B (FC) + Toollery	93.50	98.00 $\uparrow$	90.50	99.00 $\uparrow$	91.50 $\uparrow$	88.50 $\uparrow$	85.88 $\uparrow$	83.50 $\uparrow$
Qwen3-235B (FC)	98.00	93.00	87.50	89.50	85.50	80.00	81.13	80.10
Qwen3-235B (FC) + Toollery	98.50 $\uparrow$	92.50	89.00 $\uparrow$	89.50	90.00	91.50 $\uparrow$	88.63 $\uparrow$	87.30 $\uparrow$

References

- Mayank Agarwal, Ibrahim Abdelaziz, Kinjal Basu, Merve Unuvar, Luis A Lastras, Yara Rizk, and Pavan Kapanipathi. 2025. Toolrm: Outcome reward models for tool-calling large language models. *arXiv preprint arXiv:2509.11963*.
- Amazon Web Services. 2024. Boto3: The AWS SDK for Python. <https://boto3.amazonaws.com/v1/documentation/api/latest/index.html>. Accessed: 2024-05-01.
- Tiantian Gan and Qiyao Sun. 2025. Rag-mcp: Mitigating prompt bloat in llm tool selection via retrieval-augmented generation. *arXiv preprint arXiv:2505.03275*.
- Zirui Guo, Xubin Ren, Lingrui Xu, Jiahao Zhang, and Chao Huang. 2025. Rag-anything: All-in-one rag framework. *arXiv preprint arXiv:2510.12323*.
- Minghao Li, Yingxiu Zhao, Bowen Yu, Feifan Song, Hangyu Li, Haiyang Yu, Zhoujun Li, Fei Huang, and Yongbin Li. 2023. [API-bank: A comprehensive benchmark for tool-augmented LLMs](#). In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 3102–3116, Singapore. Association for Computational Linguistics.
- Jianghao Lin, Xinyuan Wang, Xinyi Dai, Menghui Zhu, Bo Chen, Ruiming Tang, Yong Yu, and Weinan Zhang. 2025. Masstool: A multi-task search-based tool retrieval framework for large language models. *arXiv preprint arXiv:2507.00487*.
- Weiwen Liu, Xu Huang, Xingshan Zeng, Xinlong Hao, Shuai Yu, Dexun Li, Shuai Wang, Weinan Gan, Zhengying Liu, Yuanqing Yu, and 1 others. 2024a. Toolace: Winning the points of llm function calling. *arXiv preprint arXiv:2409.00920*.
- Zuxin Liu, Thai Hoang, Jianguo Zhang, Ming Zhu, Tian Lan, Juntao Tan, Weiran Yao, Zhiwei Liu, Yihao Feng, Rithesh RN, and 1 others. 2024b. Apigen: Automated pipeline for generating verifiable and diverse function-calling datasets. *Advances in Neural Information Processing Systems*, 37:54463–54482.
- Guozhao Mo, Wenliang Zhong, Jiawei Chen, Xuanang Chen, Yaojie Lu, Hongyu Lin, Ben He, Xianpei Han, and Le Sun. 2025. Livemcpbench: Can agents navigate an ocean of mcp tools? *arXiv preprint arXiv:2508.01780*.
- Varatheepan Paramanayakam, Andreas Karatzas, Iraklis Anagnostopoulos, and Dimitrios Stamoulis. 2025. Less is more: Optimizing function calling for llm execution on edge devices. In *2025 Design, Automation & Test in Europe Conference (DATE)*, pages 1–7. IEEE.
- Shishir G Patil, Tianjun Zhang, Xin Wang, and Joseph E Gonzalez. 2024. Gorilla: Large language model connected with massive apis. *Advances in Neural Information Processing Systems*, 37:126544–126565.
- Yujia Qin, Shihao Liang, Yining Ye, Kunlun Zhu, Lan Yan, Yaxi Lu, Yankai Lin, Xin Cong, Xiangru Tang, Bill Qian, and 1 others. 2023. Toolllm: Facilitating large language models to master 16000+ real-world apis. *arXiv preprint arXiv:2307.16789*.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems*, 36:68539–68551.
- Zhiyong Wu, Chengcheng Han, Zichen Ding, Zhenmin Weng, Zhoumianze Liu, Shunyu Yao, Tao Yu, and Lingpeng Kong. 2024. Os-copilot: Towards generalist computer agents with self-improvement. *arXiv preprint arXiv:2402.07456*.
- Hanwen Xu, Xuyao Huang, Yuzhe Liu, Kai Yu, and Zhijie Deng. 2025. Tps-bench: Evaluating ai agents’ tool planning & scheduling abilities in compounding tasks. *arXiv preprint arXiv:2511.01527*.
- Shunyu Yao, Howard Chen, John Yang, and Karthik Narasimhan. 2022a. Webshop: Towards scalable real-world web interaction with grounded language agents. *Advances in Neural Information Processing Systems*, 35:20744–20757.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik R Narasimhan, and Yuan Cao. 2022b. React: Synergizing reasoning and acting in language models. In *The eleventh international conference on learning representations*.

YanZhao Zheng, ZhenTao Zhang, Chao Ma, YuanQiang
Yu, JiHuan Zhu, Baohua Dong, and Hangcheng
Zhu. 2026. Skillrouter: Retrieve-and-rerank skill
selection for llm agents at scale. *arXiv preprint*
arXiv:2603.22455.